Software Requirements Document

Tango - YouTube to Anki Vocabulary Pipeline

Version 0.4.0 | July 2026

---

1. Introduction

This document specifies the software requirements for Tango version 0.4.0. It covers functional behaviour, interface specifications, data requirements, and constraints. It is intended for developers working on the codebase.

2. System Interfaces

2.1 CLI Interface

The system exposes a single CLI entry point via python -m pipeline. The following flags are supported in v0.4.0:

--video-id VIDEO_ID: YouTube video ID or URL to process. Required in default mode.

--deck DECK_NAME: Anki deck name. Supports :: sub-deck notation. If omitted, interactive selection is shown.

--language LANG: BCP-47 code for subtitle language (e.g. fr). If omitted, inferred from deck name.

--def-lang LANG: BCP-47 code for definition output language. Defaults to transcript language.

--review: Process review.json decisions and build .apkg for approved words.

--process-backlog: Process the Anki backlog. Requires Anki to be running.

--list-languages: Print all supported language names and BCP-47 codes, then exit.

--verbose: Enable DEBUG logging output.

2.2 Makefile Interface

The Makefile wraps the CLI for common operations. make run requires VIDEO_ID and DECK variables. make review and make backlog require DECK. LANGUAGE and DEF_LANG are optional. make translate-setup installs argostranslate. make test runs the unit suite. make test-all runs the full suite including integration tests. make clean removes the venv, output directory, and cache files.

2.3 AnkiConnect Interface

The system communicates with AnkiConnect at ANKI_HOST (default http://localhost:8765) using the version 6 API. The following actions are used: version for

health checks, deckNames for listing decks, findNotes for querying card IDs, notesInfo for fetching card field data, and importPackage for automatic package import.

3. Data Requirements

3.1 SQLite Schema

definitions: lemma (PK, composite with language), definition, example_dict, example_dict2, synonyms (JSON), antonyms (JSON), part_of_speech, source, fetched_at

anki_backlog: lemma, deck_name (composite PK), queued_at

processed_videos: video_id (PK), processed_at, deck_name, card_count, word_count

generated_packages: id (autoincrement PK), video_id, file_path, deck_name, card_count, created_at

vocabulary: lemma, video_id (composite PK), frequency, position, part_of_speech, added_at

3.2 Card Fields

Each Anki note contains exactly ten fields in this order: Word, Class, Definition, 1st Example Sentence, 2nd Example Sentence, Example from Youtube Video, Synonyms, Antonyms, VideoID, Source. All text fields are capped at 256 characters. The genanki model ID is 1607392321. Changing this ID after initial import breaks Anki review history and must not be done without a migration plan.

3.3 Review File Schema

review.json is a JSON array. Each entry has four fields: lemma (string), matched_front (string), score (float), and decision (string or null). Valid decision values are add, skip, and null. Null means the user has not yet reviewed this entry. The pipeline processes only entries with non-null decisions.

4. Constraints and Assumptions

4.1 Rate Limits

The Merriam-Webster free tier allows 1000 requests per day. The SQLite cache prevents re-fetching words already processed. dictionaryapi.dev has no published rate limit but may throttle under heavy use. The API_DELAY environment variable controls the delay between live API calls and defaults to 0.5 seconds.

4.2 Language Model Size

Each argostranslate language pair model is approximately 100 to 200 megabytes. Models are downloaded once and stored permanently. The system will prompt the user before

downloading. Translation is optional and only required when DEF_LANG differs from LANGUAGE.

4.3 AnkiConnect Availability

AnkiConnect requires the Anki desktop application to be running. When Anki is not available, all vocabulary is written to the SQLite backlog. The user runs make backlog to process it when Anki is available. The pipeline does not hard-fail when Anki is unavailable.

4.4 spaCy Model

The en_core_web_sm model must be installed before running the pipeline. The Makefile make spacy-model target handles this. The model is loaded lazily on first call and cached for the process lifetime. POS tagging accuracy on domain-specific or non-English vocabulary may be lower than benchmarks suggest.

5. Quality Requirements

5.1 Test Coverage

All modules must have unit tests runnable without external dependencies. Tests must not make real HTTP requests, real AnkiConnect calls, or require installed translation models. Integration tests are permitted but must be gated with the pytest.mark.integration marker and excluded from the default test run.

5.2 Error Handling

The pipeline must not produce a Python traceback for any expected failure condition. API failures, missing models, unreachable services, and invalid inputs must all produce a clear user-facing error message via the CLI error formatter and exit with a non-zero code. Unexpected failures may produce tracebacks during v0.4.0 development but must be caught by v1.0.0.